

Day 13 : Room Database

1. Introduction to Flutter Floor Database

Floor is an ORM (Object Relational Mapping) library for Flutter that works on top of SQLite. Instead of writing raw SQLite code everywhere, Floor generates database code automatically using annotations.

2. Why Do We Use it?

- Less boilerplate code
- Type-safe queries
- Easy CRUD operations
- Automatic code generation
- Better for large projects
- Cleaner architecture

3. Floor vs SQLite

Feature	SQLite (sqflite)	Floor
Query Writing	Manual	Annotation based
Type Safety	No	Yes
Boilerplate	More	Less
Learning Curve	Easy	Medium
Large Projects	Difficult	Better
Performance	Similar	Similar
Code Generation	No	Yes
Maintenance	Harder	Easier

4. When we use ?

Use sqflite

- Small applications
- Learning database basics
- Simple CRUD projects

Use Floor

- Large applications
- Multiple tables
- Complex queries
- Clean architecture projects

5. Add Dependencies

Pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  
  floor: ^1.5.0  
  sqflite: ^2.3.0  
  
dev_dependencies:  
  floor_generator: ^1.5.0  
  build_runner: ^2.4.0
```

6. Create Entity Class

Entity represents a table.

```
import 'package:floor/floor.dart';  
  
@Entity(tableName: 'notes')  
class Note {  
  
  @PrimaryKey(autoGenerate: true)  
  final int? id;  
  
  final String title;  
  final String description;  
  
  Note(this.id, this.title, this.description);  
}
```

```
CREATE TABLE notes(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT,  
  description TEXT  
)
```

7 . Create DAO

DAO = Data Access Object

Used for database operations.

```
import 'package:floor/floor.dart';  
import 'note.dart';  
  
@dao  
abstract class NoteDao {  
  
  @Query('SELECT * FROM notes')  
  Future<List<Note>> getAllNotes();  
  
  @Insert()  
  Future<void> insertNote(Note note);  
  
  @Update()  
  Future<void> updateNote(Note note);  
  
  @delete  
  Future<void> deleteNote(Note note);  
}
```

Annotation	Work
@dao	DAO class
@Query	SQL Query
@Insert	Insert record
@Update	Update record
@delete	Delete record

8. Creating Database Class

```
import 'dart:async';
import 'package:floor/floor.dart';
import 'package:sqflite/sqflite.dart' as sqflite;

import 'note.dart';
import 'note_dao.dart';

part 'app_database.g.dart';

@Database(version: 1, entities: [Note])

abstract class AppDatabase extends FloorDatabase {
  NoteDao get noteDao;
}
```

9. CRUD Operation

1. Insert

```
await noteDao.insertNote(note);
```

2. Read

```
await noteDao.getAllNotes();
```

3. Update

```
await noteDao.updateNote(note);
```

4. Delete

```
await noteDao.deleteNote(note);
```

10. Relationships in Floor

One-to-many Example

Student → Multiple Courses

```
@Entity(tableName: 'student')
class Student {
  @PrimaryKey()
  final int id;

  final String name;

  Student(this.id, this.name);
}
```

```

@Entity(
    tableName: 'course',
    foreignKeys: [
        ForeignKey(
            childColumns: ['studentId'],
            parentColumns: ['id'],
            entity: Student,
        )
    ]
)
class Course {

    @PrimaryKey()
    final int id;

    final String courseName;
    final int studentId;

    Course(this.id,this.courseName,this.studentId);
}

```

11.Return Types in DAO

1. Future

```
Future<List<Note>>
```

2. Stream

```
Stream<List<Note>>
```

Example

```

@Query('SELECT * FROM notes')
Stream<List<Note>> getNotes();

```

OUTPUT :

